ЛАБОРАТОРНАЯ РАБОТА № 2 · ПАМЯТКА К ЗАЩИТЕ

# Парадигмы интерфейса: командный язык и меню

Вариант 1 — переименование файла. Дисциплина «Проектирование человеко-машинного интерфейса».

## 1. Суть работы за минуту

Одну и ту же задачу — управление файлами в каталоге — мы решили двумя способами общения человека с программой и сравнили их между собой.

- **Программа 1,** `shell.py`**.** Пользователь печатает команды. Это парадигма «командный язык».
- **Программа 2,** `menu.py`**.** Пользователь выбирает действия мышью из меню. Это парадигма «меню».
- **Обе умеют одно и то же:** показать содержимое каталога, переименовать один файл или группу файлов, сменить текущий каталог.
- **Итог сравнения:** программа с меню потребовала почти втрое больше кода, а по скорости работы каждая парадигма выигрывает в своих задачах.

## 2. Задание и вариант

Вариантов шесть: 1 — переименование файла, 2 — изменение типа, 3 — установка атрибутов, 4 — удаление, 5 — копирование, 6 — перемещение. У нас **вариант 1**. Что требовалось и чем это закрыто:

| Требование задания | Командный язык | Меню |
|---|---|---|
| Вывод информации о содержимом каталога | `dir` | список файлов в окне |
| Обработка одного файла (переименование) | `ren <имя> <новое-имя>` | «Правка → Переименовать…» |
| Обработка группы файлов по условиям пользователя | `ren <маска> <новая-маска>` | выделение файлов и «Правка → Переименовать группу…» |
| Смена текущего каталога | `cd <путь>` | двойной щелчок, строка «..», адресная строка |

Кроме программ нужно было: разработать командный язык, показать его на примерах и согласовать с преподавателем, сравнить затраты на разработку и эффективность работы пользователя, оформить отчёт.

В отчёте (файл `Отчёт_ЛР2_ЧМИ_ГОСТ.docx`) по заданию должны быть четыре части, и все они есть: постановка задачи (раздел 1), исчерпывающее описание командного языка (раздел 2), краткое описание программ в виде руководства пользователя и руководства программиста (разделы 3 и 4), вывод о пригодности парадигм к задаче (разделы 5 и 6).

## 3. Немного теории

| Понятие | Что это |
|---|---|
| Парадигма интерфейса | Общий принцип, по которому построен диалог человека с программой: как пользователь сообщает, что нужно сделать, и в каком виде получает ответ. Примеры: командный язык, меню, заполнение форм, прямое манипулирование объектами на экране |
| Командный язык | Набор команд и правил их записи. Пользователь управляет программой, вводя строки текста |
| Меню | Список доступных действий, из которого пользователь выбирает нужное. Помнить ничего не надо: варианты видны на экране |
| Виджет | Готовый элемент интерфейса из библиотеки: кнопка, поле ввода, список, меню. В работе использована библиотека Tkinter, она входит в Python |
| Текущий каталог | Папка, в которой программа работает сейчас. Все команды относятся к ней |
| Относительный и абсолютный путь | Относительный отсчитывается от текущего каталога: `Фото`, `..`. Абсолютный — полный путь от диска: `C:\Windows` |
| Маска | Образец имени со знаками `*` (любое число любых символов) и `?` (ровно один символ). Маской задают группу файлов: `*.txt` — все файлы с расширением txt |

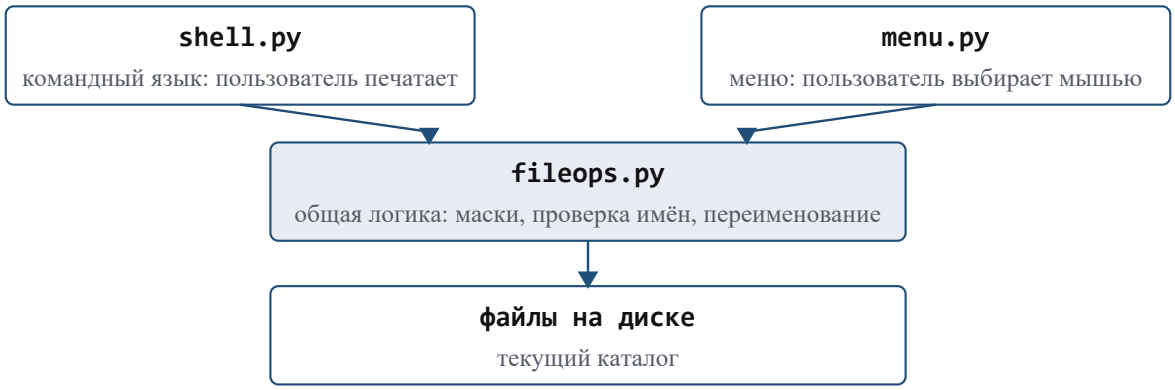
Определение парадигмы стоит сверить с конспектом лекции: преподаватель может ждать свою формулировку.

## 4. Что сделано: файлы и где что лежит

Всё находится в папке `lab2`, отчёт — уровнем выше. Главных файлов три: общая логика и по одному файлу на каждую программу.

```
hmi-design/
├─ Отчёт_ЛР2_ЧМИ_ГОСТ.docx     отчёт
└─ lab2/
   ├─ fileops.py              общая логика, интерфейса в ней нет
   ├─ shell.py                программа 1: командный язык
   ├─ menu.py                 программа 2: меню
   ├─ make_demo.py            создаёт тестовую папку demo
   ├─ test_fileops.py         тесты общей логики
   ├─ README.md               краткое описание
   └─ demo/                   тестовая папка, создаётся скриптом
```

| Файл | Строк | Назначение |
|---|---|---|
| `fileops.py` | 287 | Чтение каталога, маски, проверка и выполнение переименования. Сам не запускается — его подключают обе программы |
| `shell.py` | 231 | Программа с командным языком: читает строку, разбирает её и выполняет команду |
| `menu.py` | 690 | Программа с меню: окно со списком файлов, меню и диалоговые окна |
| `make_demo.py` | 94 | Создаёт папку `demo`: 12 файлов и 3 каталога (в них ещё 7 файлов). Повторный запуск возвращает папку в исходное состояние |
| `test_fileops.py` | 169 | 17 автоматических тестов общей логики |




Обе программы — это только разный интерфейс. Проверяет имена и переименовывает один и тот же код из `fileops.py`, поэтому правила и сообщения об ошибках у программ одинаковые.

## 5. Как запустить

Нужен Python версии 3.10 или новее, устанавливать больше ничего не надо. Программы рассчитаны на Windows. В терминале:

```
cd lab2
python make_demo.py                создать папку demo или вернуть её в исходное состояние
python shell.py demo               программа с командным языком
python menu.py demo                программа с меню, откроется окно
python -m unittest test_fileops    тесты, необязательно
```

Слово `demo` в конце — папка, с которой программа начнёт работу. Без него программа откроет ту папку, из которой запущена, то есть `lab2`, и можно случайно переименовать файлы с кодом.

## 6. Программа с командным языком

После запуска появляется приглашение с путём текущего каталога. Вводите команду и нажимаете Enter. Строка — это команда и параметры через пробел. Имя с пробелами берётся в кавычки, регистр букв не важен.

| Команда | Что делает |
|---|---|
| `dir` | Показывает содержимое текущего каталога: имя, размер, время изменения |
| `dir <маска>` | Показывает только то, что подходит под маску. Так проверяют маску перед переименованием |
| `cd <путь>` | Переходит в каталог; путь относительный или абсолютный. `cd ..` — на уровень выше, `cd` без параметра — показать текущий каталог |
| `ren <имя> <новое-имя>` | Переименовывает один файл |
| `ren <маска> <новая-маска>` | Переименовывает группу файлов |
| `help`, `help <команда>` | Список команд; справка по одной команде с примерами |
| `exit` | Выход |

### Как работает новая маска

Знак * в новой маске подставляет то, что совпало со знаком * в исходной маске, знак ? — то, что совпало с ?. Всё остальное в новой маске пишется как есть.

| Команда | Было → стало | Смысл |
|---|---|---|
| `ren *.txt *.md` | `план.txt` → `план.md` | сменить расширение |
| `ren * old_*` | `отчёт_1.docx` → `old_отчёт_1.docx` | добавить текст в начало |
| `ren *.* *_2026.*` | `отчёт_1.docx` → `отчёт_1_2026.docx` | добавить текст в конец, перед расширением |
| `ren IMG_*.jpg отпуск_*.jpg` | `IMG_0001.jpg` → `отпуск_0001.jpg` | заменить начало имени |
| `ren отчёт_?.docx отчёт_2026_?.docx` | `отчёт_3.docx` → `отчёт_2026_3.docx` | вставить текст в середину |

### Пример работы

```
Управление файловым каталогом. Командный язык.
Введите help, чтобы увидеть список команд.

...\lab2\demo> dir *.csv
Каталог ...\lab2\demo, маска *.csv

Имя               Размер  Изменён
data_2024.csv    47,2 КБ  14.01.2025 10:20
data_2025.csv    50,9 КБ  12.01.2026 09:45

Подходит под маску: 2 файла.

...\lab2\demo> ren data_*.csv stat_*.csv
data_2024.csv → stat_2024.csv
data_2025.csv → stat_2025.csv
Переименовано 2 файла.

...\lab2\demo> ren *.txt *.md
Файл с именем readme.md уже существует. Переименование отменено.

...\lab2\demo> cd Фото

...\lab2\demo\Фото> ren *.jpg *_trip.jpg
IMG_0001.jpg → IMG_0001_trip.jpg
IMG_0002.jpg → IMG_0002_trip.jpg
IMG_0003.jpg → IMG_0003_trip.jpg
IMG_0004.jpg → IMG_0004_trip.jpg
Переименовано 4 файла.

...\lab2\demo\Фото> cd ..

...\lab2\demo> ren заметки.txt записи.txt
заметки.txt → записи.txt
Переименован 1 файл.

...\lab2\demo> exit
```

### Правило «всё или ничего»

Перед переименованием программа проверяет все новые имена. Если хотя бы одно занято или недопустимо, не переименовывается ни один файл — как в примере выше с `ren *.txt *.md`. Если сбой случился уже в процессе (например, файл открыт в другой программе), переименованным файлам возвращаются прежние имена. Каталоги команда `ren` не переименовывает.

### Основные сообщения об ошибках

| Ситуация | Сообщение |
|---|---|
| Неизвестная команда | `Неизвестная команда: <глагол>. Введите help.` |
| Файла нет | `Файл не найден: <имя>` |
| Каталога нет | `Каталог не найден: <путь>` |
| Ничего не подошло под маску | `Нет файлов, соответствующих маске: <маска>` |
| Новое имя уже занято | `Файл с именем <новое-имя> уже существует. Переименование отменено.` |
| Два файла получили бы одно имя | `Файлы <имя-1> и <имя-2> получили бы одно имя <новое-имя>. Переименование отменено.` |
| Вместо файла указан каталог | `Это каталог, а не файл: <имя>` |
| В имени запрещённые символы | `Недопустимое имя файла: <новое-имя>. Нельзя использовать символы \ / : * ? " < > \|` |

Первые пять — из таблицы ошибок в PDF с вашим вариантом. Полный список сообщений — таблица 5 отчёта.

## 7. Программа с меню

Окно сверху вниз: строка меню, панель кнопок с адресной строкой, список файлов текущего каталога, строка состояния. Пункты, которые сейчас недоступны, серые: например, «Переименовать…» работает, только когда выделен ровно один файл.

| Где | Что там есть |
|---|---|
| Меню «Файл» | Открыть каталог… (Ctrl+O), Вверх (Backspace), Обновить (F5), Выход (Alt+F4) |
| Меню «Правка» | Переименовать… (F2), Переименовать группу… (Ctrl+R), Выделить всё (Ctrl+A), Выделить по маске… (Ctrl+F), Снять выделение (Esc) |
| Меню «Вид» | Флажки столбцов «Размер» и «Изменён»; подменю «Сортировка»: по имени, по размеру, по дате изменения, по убыванию |
| Меню «Справка» | Как пользоваться (F1), О программе |
| Контекстное меню (правая кнопка мыши) | Открыть каталог, Переименовать…, Переименовать группу…, Выделить всё, Выделить по маске…, Обновить |
| Панель кнопок | Вверх, Обновить, Переименовать, Группа…, адресная строка и кнопка «Перейти» |

### Как работать

- **Ходить по каталогам.** Двойной щелчок по каталогу или Enter — войти. Строка «..» или Backspace — на уровень выше. Путь можно ввести в адресную строку.
- **Переименовать один файл.** Выделить файл, выбрать «Правка → Переименовать…» (или F2, или двойной щелчок), ввести новое имя, Enter. Часть имени до расширения уже выделена — можно сразу печатать.
- **Выделить группу.** Мышью с Ctrl (по одному) или Shift (подряд), Ctrl+A — все. «Выделить по маске…» выделяет по образцу и сразу показывает, сколько файлов подходит.
- **Переименовать группу.** «Правка → Переименовать группу…», выбрать действие, ввести текст, посмотреть предпросмотр и нажать «Переименовать».

В окне группы три действия. На примере файла `readme.txt`:

| Действие | Что ввести | Получится |
|---|---|---|
| добавить текст в начало имени | `old_` | `old_readme.txt` |
| добавить текст в конец имени (перед расширением) | `_2026` | `readme_2026.txt` |
| сменить расширение | `md` | `readme.md` |

Действие применяется сразу ко всем выделенным файлам. Таблица «Предпросмотр» показывает старые и новые имена ещё до переименования. Если новое имя занято или недопустимо, под таблицей появляется красное сообщение, а кнопка «Переименовать» недоступна.

## 8. Что внутри кода

Ниже — что есть в каждом файле. Имена те же, что в коде: любое можно найти в редакторе поиском (Ctrl+F).

### fileops.py — общая логика

| Имя | Что делает |
|---|---|
| `Entry` | Один объект каталога: имя, путь, файл это или каталог, размер, время изменения |
| `Rename` | Пара «старое имя → новое имя». Список таких пар — план переименования |
| `FileError` | Ошибка с готовым текстом сообщения для пользователя |
| `list_directory` | Читает каталог и возвращает его объекты: сначала каталоги, затем файлы |
| `resolve_directory` | Из введённого пути получает настоящий каталог или сообщает, что его нет |
| `mask_regex`, `matches`, `select` | Переводят маску в регулярное выражение и отбирают подходящие объекты |
| `substitute` | Строит новое имя по исходной и новой маске |
| `plan_rename` | Составляет план для команды `ren`: один файл или группа |
| `add_prefix`, `add_suffix`, `change_extension` | Три действия окна группы: текст в начало, текст в конец, новое расширение |

| Имя | Что делает |
|---|---|
| `validate_name` | Проверяет, что имя допустимо в Windows |
| `check_plan` | Проверяет весь план: нет ли занятых, недопустимых или совпадающих имён |
| `apply_plan` | Переименовывает файлы; при сбое возвращает прежние имена |
| `format_size`, `format_date`, `files_text` и другие | Готовят текст для вывода: «47,2 КБ», «2 файла» |

### shell.py — командный язык

| Имя | Что делает |
|---|---|
| `COMMANDS`, `SUMMARY`, `HELP` | Список команд и тексты справки |
| `split_words` | Делит введённую строку на слова с учётом кавычек |
| `print_entries` | Печатает таблицу файлов для команды `dir` |
| `Shell.run` | Главный цикл: показать приглашение, прочитать строку, выполнить |
| `Shell.execute` | По первому слову находит команду и вызывает её метод; ошибку печатает как сообщение |
| `do_dir`, `do_cd`, `do_ren`, `do_help`, `do_exit` | По одному методу на каждую команду |
| `main` | Запуск: определяет начальный каталог и создаёт `Shell` |

### menu.py — меню

| Имя | Что делает |
|---|---|
| `App` | Главное окно программы |
| `build_menu`, `build_toolbar`, `build_list`, `build_status` | Строят строку меню и контекстное меню, панель кнопок, список файлов, строку состояния |
| `open_directory`, `show_entries` | Читают каталог и заполняют список |
| `on_select` | Включает и отключает пункты меню и кнопки в зависимости от выделения |
| `rename_one`, `rename_group`, `apply` | Открывают окна переименования и выполняют его |
| `select_by_mask`, `go_up`, `go_to_address` | Выделение по маске и переходы по каталогам |
| `Dialog` | Общая основа всех окон: кнопки, строка сообщения, показ по центру |
| `RenameDialog`, `MaskDialog`, `GroupDialog`, `HelpWindow` | Окна: новое имя файла, выделение по маске, действие над группой, справка |
| `ACTIONS` | Список действий окна группы |

## Путь одной команды: ren *.txt *.md

1. `Shell.run` читает строку, `split_words` делит её на слова: `ren`, `*.txt`, `*.md`.

2. `Shell.execute` видит команду `ren` и вызывает метод `do_ren`.
3. `list_directory` читает текущий каталог.
4. `plan_rename` отбирает файлы по маске и для каждого строит новое имя через `substitute`.
5. `check_plan` проверяет все новые имена. Если есть проблема — выводится ошибка, файлы не тронуты.
6. `apply_plan` переименовывает файлы.
7. `do_ren` печатает пары «старое → новое» и число переименованных файлов.

В программе с меню шаги 3, 5 и 6 те же самые. Отличается только начало: вместо разбора строки план составляет окно `GroupDialog` из выбранного действия и введённого текста.

## 9. Сравнение парадигм

### Затраты на разработку

| Программа | Файл | Строк кода | Функций и методов |
|---|---|---|---|
| Командный язык | `shell.py` | 204 | 12 |
| Меню | `menu.py` | 588 | 55 |
| Общая логика | `fileops.py` | 214 | 24 |

Строки посчитаны без пустых строк и комментариев. Меню почти втрое больше: нужно построить главное окно, четыре диалоговых окна и следить за состоянием пунктов меню. В командном языке кода мало, зато больше труда уходит на придумывание самого языка: синтаксис, правила масок, тексты сообщений и справки.

### Эффективность работы пользователя: модель KLM

KLM (Keystroke-Level Model) оценивает время работы опытного пользователя без ошибок. Действие раскладывают на элементарные операции и складывают их время.

| Операция | Время | Что означает |
|---|---|---|
| K | 0,20 с | нажатие клавиши |
| P | 1,10 с | наведение указателя мыши на цель |
| B | 0,10 с | нажатие или отпускание кнопки мыши |
| H | 0,40 с | перенос руки между клавиатурой и мышью |
| M | 1,35 с | мысленная подготовка |

Пример: команда `ren readme.txt manual.txt` — это 3 раза подумать и 26 нажатий (25 символов и Enter): $3M + 26K = 3 \cdot 1{,}35 + 26 \cdot 0{,}20 = 9{,}25$ с.

| Задача | Командный язык | Меню | Вывод |
|---|---|---|---|
| 1. Переименовать один файл | 9,25 с | 9,75 с | почти одинаково |
| 2. Сменить расширение всех файлов .txt | 7,65 с | 16,65 с | команда быстрее в 2,2 раза |
| 3. Добавить текст в конец имён всех файлов каталога | 8,65 с | 13,70 с | команда быстрее в 1,6 раза |
| 4. Добавить приставку к двум файлам без общей маски | 22,10 с | 12,40 с | меню быстрее в 1,8 раза |
| 5. Перейти в каталог и вернуться | 9,00 с | 5,70 с | меню быстрее в 1,6 раза |
| 6. Узнать, какие файлы подходят под маску | 4,90 с | 7,10 с | команда быстрее в 1,4 раза |
| **Итого** | **61,55 с** | **65,30 с** | |

Это расчёт по модели, а не замеры с секундомером: время на обучение и на исправление опечаток не учитывается.

### Выводы

- **Командный язык** быстрее в групповых операциях, которые описываются правилом (маской), и удобен опытному пользователю. Минусы: команды и маски нужно помнить, имена печатаются полностью и без опечаток, результат не виден до выполнения.
- **Меню** не требует ничего помнить: действия видны, недоступные затенены, произвольные файлы выбираются мышью, предпросмотр показывает результат заранее. Удобно новичку. Минусы: операции «по правилу» требуют нескольких окон, а кода почти втрое больше.
- **Лучше всего сочетать:** меню и список — для просмотра и отдельных файлов, команды — для массового переименования.
- **Различие в возможностях:** заменить часть имени у группы файлов (`ren IMG_*.jpg отпуск_*.jpg`) можно только командой. В окне группы три действия: текст в начало, текст в конец, смена расширения.

## 10. Контрольные вопросы

Три вопроса со слайда «Контрольные вопросы (1/2)». Второго слайда с вопросами среди фотографий не было.

### Дайте определение термину «парадигма» применительно к интерфейсам программных систем

Парадигма интерфейса — общий принцип организации диалога человека с программой: каким способом пользователь сообщает, что нужно сделать, и как получает ответ. В работе реализованы две парадигмы: командный язык и меню.

### Что такое командный язык? Опишите его назначение и основные компоненты

Командный язык — набор команд и правил их записи, с помощью которого пользователь управляет программой, вводя строки текста. Назначение — быстро и точно задавать действия, в том числе сразу над группой объектов. Компоненты на примере нашей работы:

- **Словарь:** команды `dir`, `cd`, `ren`, `help`, `exit` и специальные знаки `*`, `?`, кавычки.
- **Синтаксис** — правила записи: `<команда> { <пробел> <параметр> }`, имя с пробелами в кавычках.
- **Смысл команд** — что делает каждая: `dir` показывает, `cd` переходит, `ren` переименовывает.
- **Диалог:** приглашение с текущим каталогом, сообщения о результате и об ошибках, справка `help`.

### Какие типы меню вам известны? Какие способы их организации можно предложить и какие использованы в работе?

- **Типы:** строка меню (главное меню), выпадающее меню, каскадное подменю, контекстное (всплывающее) меню, панель кнопок.
- **Виды пунктов:** команда, флажок (включено или выключено), переключатель (один из нескольких).
- **Способы организации:** группировка пунктов по смыслу с разделителями, иерархия (меню → подменю), порядок по частоте использования, сочетания клавиш для частых действий, затенение недоступных пунктов.
- **В работе использовано всё перечисленное:** строка меню с четырьмя выпадающими меню, каскадное подменю «Сортировка», контекстное меню списка, панель кнопок; флажки столбцов и переключатели сортировки.

### Вопросы, которые могут задать по программам

Отвечать лучше своими словами, здесь только суть.

| Вопрос | Короткий ответ |
|---|---|
| Почему каталоги не переименовываются? | Вариант — переименование файла. Под маску отбираются только файлы, а на имя каталога программа отвечает «Это каталог, а не файл» |
| Что будет, если новое имя занято? | Сообщение об ошибке, и ни один файл не переименуется |
| Зачем команде `dir` маска? | Чтобы заранее проверить, какие файлы попадут под маску, до переименования |
| Зачем общий модуль `fileops.py`? | Обе программы работают по одним правилам, код не дублируется. Сравнение честное: отличается только интерфейс |
| Какие виджеты использованы? | `tk.Menu`, `ttk.Treeview`, `ttk.Button`, `ttk.Entry`, `ttk.Label`, `ttk.Radiobutton`, `ttk.LabelFrame`, `ttk.Scrollbar`, `tk.Text`, `tk.Toplevel`, окна `messagebox` и `filedialog` |
| Как оценивали скорость работы? | По модели KLM: сумма времени элементарных операций |
| Можно ли дать файлам общее имя с номерами? | Нет, нумерации в программах нет: каждый файл сохраняет своё имя как основу |